

基于迅为开发板的安卓应用控制 LED 灯泡点 亮项目开题报告

姓 名：顾瑞奇

学 号：2305221231

指导教师姓名：

专 业 名 称：软件工程

研 究 方 向：工业软件开发

目录

一、项目概述	3
1.1 项目背景	3
1.2 项目目标	3
二、技术原理与相关知识	5
2.1 安卓应用开发基础	5
2.2 迅为开发板介绍	6
2.3 LED 灯泡控制原理	6
2.4 三者结合的工作机制	7
三、系统设计方案	8
3.1 硬件设计	8
3.2 软件设计	8
四、关键技术难点及解决方案	11
4.1 通信稳定性问题	11
4.2 权限管理问题	11
4.3 硬件兼容性问题	12
五、预期成果	13
5.1 实现功能	13
5.2 性能指标	13
六、进度安排	15
6.1 需求分析与方案设计阶段	15
6.2 硬件搭建与软件开发阶段	15
6.3 测试与优化阶段	15
6.4 项目验收阶段	16
七、参考文献	17
[1] 明日科技. Android 从入门到精通 [M]. 清华大学出版社, 2020.	17
[2] 李宁. 疯狂 Android 讲义 [M]. 电子工业出版社, 2021.	17
[3] 迅为电子. iTOP-XX 开发板用户手册 [Z]. 迅为电子有限公司, 2022.	17
[4] 韦东山. 嵌入式 Linux 应用开发完全手册 [M]. 人民邮电出版社, 2019.	17
[5] 孙鑫. Java 核心技术 [M]. 电子工业出版社, 2020.	17

一、项目概述

1.1 项目背景

随着科技的飞速发展，智能家居已经逐渐走进人们的生活，成为现代生活中不可或缺的一部分。智能家居通过物联网技术，将各种家居设备连接在一起，实现设备的智能化控制和管理，为用户提供更加便捷、舒适、安全的生活体验。在智能家居系统中，灯光控制是一个基础且重要的功能。传统的 LED 灯泡控制方式通常依赖于物理开关，这种方式在操作上存在一定的局限性，无法满足人们对于便捷性和智能化的追求。而利用安卓应用来控制 LED 灯泡，能够打破空间限制，用户可以通过手机、平板电脑等安卓设备，随时随地对 LED 灯泡进行开关、调光等操作。

安卓系统凭借其开放性、广泛的用户基础以及强大的应用生态系统，在智能家居控制领域具有巨大的优势。基于安卓平台开发的智能家居应用能够方便地与各种智能硬件设备进行交互，为用户提供统一、便捷的控制界面。通过将安卓应用与 LED 灯泡相结合，可以实现更加智能化、个性化的灯光控制。例如，用户可以根据不同的场景需求，通过安卓应用设置不同的灯光亮度和颜色，营造出舒适的氛围；还可以设置定时开关功能，实现自动化的灯光控制，提高能源利用效率。

此外，对于开发者来说，基于安卓平台开发 LED 灯泡控制应用，不仅能够提升自身的技术能力，还能够深入了解物联网技术在智能家居领域的应用，积累项目开发经验。同时，开发这样的应用也有助于推动智能家居技术的发展，满足市场对于智能家居产品的需求。

1.2 项目目标

本项目旨在基于迅为开发板，实现一个安卓应用来控制 LED 灯泡的点亮功能。通过完成该项目，将达成以下具体目标：

- **搭建开发环境：**成功搭建基于迅为开发板的安卓开发环境，包括安装必要的开发工具（如 Android Studio、Java 开发工具包 JDK 等），配置开发板的驱动程序和相关开发环境变量，确保开发板与电脑之间能够正常通信，为后续的开发工作奠定基础。

- **硬件连接与驱动开发：**完成迅为开发板与 LED 灯泡之间的硬件连接，根据硬件连接方式和开发板的特性，开发相应的设备驱动程序，实现开发板对 LED 灯泡的硬件控制。确保驱动程序能够稳定、准确地控制 LED 灯泡的点亮和熄灭，为安卓应用提供可靠的硬件控制接口。
- **安卓应用开发：**开发一个功能简洁、界面友好的安卓应用，实现通过该应用向迅为开发板发送控制指令，从而控制 LED 灯泡的点亮状态。应用将具备基本的用户界面，包括开关按钮等，方便用户操作。同时，应用将实现与开发板之间的通信功能，确保控制指令能够准确无误地传输到开发板上。
- **系统测试与优化：**对整个系统进行全面的测试，包括功能测试、稳定性测试、兼容性测试等。通过测试，发现并解决系统中存在的问题，对系统进行优化，提高系统的性能和稳定性。确保系统在各种情况下都能够正常工作，满足用户的需求。

二、技术原理与相关知识

2.1 安卓应用开发基础

安卓应用开发基于 Java 或 Kotlin 语言，采用 Model - View - Presenter（MVP）、Model - View - ViewModel（MVVM）等架构模式，以实现代码的高内聚、低耦合，提高可维护性和可扩展性。其基本组件包括：

- **Activity**: 是应用与用户交互的可视化界面，一个应用可包含多个 Activity，它们之间通过 Intent 进行通信和数据传递，以此实现页面跳转和业务逻辑的衔接。例如，在一个电商应用中，用户从商品列表 Activity 跳转到商品详情 Activity，就需要通过 Intent 传递商品的相关信息。
- **Service**: 在后台运行，不与用户直接交互，主要用于执行如网络请求、数据下载、音乐播放等长时间运行的任务，以确保这些任务在后台稳定执行，不影响用户对其他界面的操作。以音乐播放应用为例，当用户切换到其他应用或锁屏时，音乐播放 Service 仍能继续运行。
- **BroadcastReceiver**: 用于接收系统或其他应用发送的广播消息，如电量变化、网络状态改变等，使应用能够及时响应这些系统事件，做出相应的处理。比如，当网络状态发生变化时，应用可以通过 BroadcastReceiver 接收到广播，提示用户网络已连接或断开。
- **ContentProvider**: 实现不同应用之间的数据共享，通过 URI 来唯一标识其数据集，其他应用通过 ContentResolver 类来访问其数据，为应用间的数据交互提供了统一的接口和规范。像系统的通讯录应用，就通过 ContentProvider 将联系人数据共享给其他需要访问联系人信息的应用。

开发环境搭建方面，Android Studio 是官方推荐的集成开发环境（IDE），功能强大，集成了代码编写、调试、界面可视化设计、性能分析等功能，可实现从项目创建到最终打包的全流程开发。安装 Android Studio 后，还需安装 Java 开发工具包

（JDK），因为安卓应用开发基于 Java 语言，JDK 提供了编译、运行 Java 程序的环境和工具。同时，需要配置 Android SDK（软件开发工具包），它包含了开发安卓应用所需的 API 库、工具和资源，如安卓系统的各种版本的 SDK、构建工具、调试工具

等。在 **Android Studio** 中，可以方便地管理和更新 **SDK**，确保开发环境与最新的安卓系统版本兼容。

2.2 迅为开发板介绍

迅为开发板是一款功能丰富、性能强大的开发平台，具备多种硬件资源和接口，为各类应用开发提供了有力支持。其硬件资源包括高性能的处理器，能够快速处理各种数据和指令，满足复杂应用的计算需求；大容量的内存，可高效存储和读取数据，保障系统和应用的流畅运行；以及一定容量的存储，用于保存系统文件、应用程序和用户数据等。

在接口方面，迅为开发板拥有多种类型的接口。**GPIO**（通用输入输出）接口是其重要的接口之一，通过这些接口可以与外部设备进行简单的数字信号交互，实现对外部设备的控制和数据采集。例如，将 **LED** 灯泡连接到 **GPIO** 接口，就可以通过开发板控制 **LED** 灯泡的亮灭。串口通信接口用于与其他设备进行串行数据传输，可实现开发板与电脑、传感器、其他微控制器等设备之间的通信，常用于调试和数据交互。网络接口使开发板能够连接到网络，实现网络通信功能，可用于远程控制、数据上传下载、与服务器交互等。此外，还可能配备 **USB** 接口、**HDMI** 接口、**SD** 卡接口等，以满足不同的应用需求，如通过 **USB** 接口连接外部设备，通过 **HDMI** 接口输出视频信号到显示器，通过 **SD** 卡接口扩展存储容量等。

在系统支持方面，迅为开发板通常支持多种操作系统，如 **Linux**、**Android** 等。不同的操作系统适用于不同的应用场景，开发者可以根据项目需求选择合适的操作系统进行开发。例如，**Linux** 系统具有开源、稳定、可定制性强等特点，适合用于对系统性能和定制化要求较高的项目；**Android** 系统则在移动应用开发和物联网应用中具有广泛的应用，其丰富的应用生态和友好的用户界面，便于开发出面向普通用户的智能应用。

2.3 LED 灯泡控制原理

LED（发光二极管）是一种能够将电能转化为光能的半导体器件，具有功耗低、寿命长、响应速度快等优点，在照明和电子设备指示等领域得到了广泛应用。其工作原理基于半导体的 **PN** 结特性，当在 **PN** 结两端施加正向电压时，电子和空穴会在 **PN** 结处复合，释放出能量，以光子的形式发射出来，从而实现发光。

通过 GPIO 控制 LED 灯泡亮灭的电气原理如下：GPIO 接口可以配置为输出模式，当配置为输出模式时，其引脚可以输出高电平（通常为 3.3V 或 5V，具体取决于开发板的电压标准）或低电平（通常为 0V）。将 LED 灯泡的一端连接到 GPIO 引脚，另一端连接到地（GND）。当 GPIO 引脚输出高电平时，电流从 GPIO 引脚流出，经过 LED 灯泡，形成回路，LED 灯泡中有电流通过，从而点亮；当 GPIO 引脚输出低电平时，回路中没有电流，LED 灯泡熄灭。为了保护 LED 灯泡和开发板，通常会在电路中串联一个限流电阻，以限制通过 LED 灯泡的电流，防止电流过大损坏 LED 灯泡或开发板。例如，根据欧姆定律 $I = (VCC - V_{LED}) / R$ （其中 I 为电流， VCC 为电源电压， V_{LED} 为 LED 灯泡的正向导通电压， R 为限流电阻的阻值），可以计算出合适的限流电阻阻值，以确保 LED 灯泡在正常工作电流范围内工作。

2.4 三者结合的工作机制

安卓应用、迅为开发板和 LED 灯泡之间通过以下方式协同工作：在安卓应用端，开发者利用安卓开发框架和相关技术，创建用户界面，用于接收用户的操作指令，如点击开关按钮。当用户在安卓应用界面上进行操作时，应用会捕获这些操作事件，并根据预设的逻辑生成相应的控制指令。这些控制指令通过安卓应用与迅为开发板之间的通信通道发送出去。

通信通道的建立可以基于多种方式，例如通过串口通信、网络通信（如 Wi-Fi、蓝牙等）。如果采用串口通信，安卓应用需要通过串口通信库（如 Android SerialPort API）来实现与开发板的串口连接，将控制指令以字节流的形式发送到串口。开发板接收到控制指令后，由其内部的处理器对指令进行解析。

在开发板端，驱动程序负责将接收到的控制指令转化为对 GPIO 接口的操作。驱动程序是开发板操作系统与硬件之间的桥梁，它了解硬件的工作原理和接口规范，能够根据控制指令对 GPIO 接口进行配置和操作。如果控制指令是点亮 LED 灯泡，驱动程序会将对应的 GPIO 引脚配置为输出模式，并输出高电平；如果是熄灭 LED 灯泡，则输出低电平。GPIO 接口的电平变化通过电路连接直接影响 LED 灯泡的工作状态，从而实现了通过安卓应用控制 LED 灯泡点亮的功能。整个过程中，安卓应用提供用户交互界面和控制逻辑，迅为开发板作为中间桥梁实现指令的传输和硬件控制，LED 灯泡则作为最终的执行设备响应控制指令，三者协同工作，共同完成了智能化的灯光控制任务。

三、系统设计方案

3.1 硬件设计

硬件设计主要是实现 LED 灯泡与迅为开发板之间的连接，利用开发板的 GPIO 接口来控制 LED 灯泡的亮灭。选择迅为开发板上合适的 GPIO 引脚，通过杜邦线将其与 LED 灯泡的控制端相连。为了保护开发板和 LED 灯泡，需要在电路中串联一个合适阻值的限流电阻，以限制通过 LED 灯泡的电流，防止电流过大损坏设备。根据 LED 灯泡的额定工作电流和开发板 GPIO 引脚的输出电压，计算限流电阻的阻值。例如，如果 LED 灯泡的额定工作电流为 20mA，开发板 GPIO 引脚输出高电平时电压为 3.3V，LED 灯泡的正向导通电压为 2V，根据欧姆定律 $R = (VCC - V_{LED}) / I$ （其中 VCC 为电源电压，VLED 为 LED 灯泡的正向导通电压，I 为电流），可计算出限流电阻阻值约为 65Ω，实际可选择与之接近的标准阻值电阻，如 68Ω。将 LED 灯泡的另一端连接到开发板的接地引脚（GND），形成完整的回路。这样，通过控制开发板 GPIO 引脚的电平状态，就可以实现对 LED 灯泡亮灭的控制。

3.2 软件设计

3.2.1 安卓应用界面设计

安卓应用的界面设计以简洁、易用为原则，主要包含一个用于控制 LED 灯泡的开关按钮。在布局文件（如 activity_main.xml）中，使用 LinearLayout 作为根布局，设置其方向为垂直方向，以方便元素的排列。在 LinearLayout 中添加一个 Button 组件，设置其 id 为 “btn_led_control”，以便在代码中能够准确引用。为 Button 设置合适的宽度和高度，如 “wrap_content”，使其大小能够自适应文字内容。设置 Button 的文本为 “控制 LED 灯泡”，让用户能够清晰地了解其功能。同时，通过设置 Button 的背景颜色、文字颜色和大小等属性，来提升界面的美观度和可读性。例如，可以设置背景颜色为蓝色，文字颜色为白色，文字大小为 18sp。此外，为了增强用户体验，当用户点击按钮时，可以添加动画效果，如按钮的缩放动画，让用户能够直观地感受到操作的反馈。在布局设计过程中，充分考虑不同屏幕尺寸和分辨率的兼容性，使用相对布局和适配单位（如 dp、sp），确保界面在各种安卓设备上都能正确显示，布局合理。

3.2.2 安卓应用逻辑设计

安卓应用的逻辑设计主要实现控制指令的发送以及与开发板的通信功能。在 **Activity** 的代码（如 **MainActivity.java**）中，首先通过 **findViewById** 方法获取布局文件中的开关按钮实例。为按钮设置点击事件监听器，当用户点击按钮时，触发相应的逻辑处理。在点击事件处理方法中，根据按钮当前的文本状态来判断 **LED** 灯泡的控制指令。如果按钮文本为“打开 **LED** 灯泡”，则表示用户希望打开 **LED** 灯泡，此时将控制指令设置为“**on**”，并更新按钮文本为“关闭 **LED** 灯泡”；反之，如果按钮文本为“关闭 **LED** 灯泡”，则将控制指令设置为“**off**”，并更新按钮文本为“打开 **LED** 灯泡”。

在与开发板的通信方面，根据选择的通信方式（如串口通信、网络通信），使用相应的通信库来实现数据传输。以串口通信为例，使用 **Android SerialPort API** 库。在应用中创建一个串口通信类（如 **SerialPortHelper**），在该类中初始化串口连接，设置串口的波特率、数据位、停止位等参数。在按钮点击事件处理方法中，通过串口通信类将控制指令以字节流的形式发送到开发板。在发送数据时，需要进行异常处理，捕获可能出现的 **IO** 异常，如串口连接失败、数据发送失败等，并在捕获到异常时，通过 **Toast** 提示用户相关错误信息，以使用户了解操作失败的原因。同时，为了确保数据发送的稳定性和可靠性，可以添加重试机制，当数据发送失败时，尝试重新发送一定次数，若多次重试仍失败，则提示用户检查设备连接和通信设置。

3.2.3 开发板端驱动程序设计

开发板端驱动程序负责将安卓应用发送的控制指令转化为对 **GPIO** 接口的操作，从而实现了对 **LED** 灯泡的控制。以 **Linux** 系统下的驱动程序开发为例，使用设备树（**Device Tree**）来描述硬件设备的信息。在设备树文件中，定义与 **LED** 灯泡连接的 **GPIO** 引脚的相关属性，包括引脚编号、复用功能等。加载驱动模块时，驱动程序会解析设备树，获取 **GPIO** 引脚的配置信息，并使用内核提供的 **GPIO** 操作函数对引脚进行初始化配置，将其设置为输出模式。

当开发板接收到安卓应用发送的控制指令后，驱动程序会根据指令内容对 **GPIO** 引脚进行相应的操作。如果接收到的指令为“**on**”，则通过 **GPIO** 操作函数将对应的 **GPIO** 引脚输出高电平，使 **LED** 灯泡点亮；如果接收到的指令为“**off**”，则将 **GPIO** 引脚输出低电平，使 **LED** 灯泡熄灭。在驱动程序中，还需要添加对指令解析的逻辑，确保能够正确识别和处理接收到的控制指令。同时，为了提高驱动程序的稳定性和可靠性，需要进行边界检查和错误处理。例如，检查接收到的指令是否合法，若指令不合法，

则返回错误信息，避免对硬件设备进行错误的操作。此外，还可以添加日志记录功能，记录驱动程序的运行状态和关键操作，以便在出现问题时能够快速定位和排查故障。

四、关键技术难点及解决方案

4.1 通信稳定性问题

在通过蓝牙或 Wi-Fi 进行通信时，可能会出现连接中断、数据丢失或延迟等不稳定情况。这主要是由于信号干扰、距离限制、设备兼容性等因素导致。例如，在蓝牙通信中，当周围存在其他 2.4GHz 频段的设备（如 Wi-Fi 路由器、微波炉等）时，会对蓝牙信号产生干扰，导致通信不稳定。

为解决这一问题，可采取以下措施：在蓝牙通信方面，使用蓝牙信号增强器来增强信号强度，减少信号衰减；优化蓝牙设备的放置位置，尽量避免与其他干扰源距离过近；同时，在代码层面，采用重传机制，当发送数据后在一定时间内未收到确认信息时，自动重发数据，确保数据的可靠传输。对于 Wi-Fi 通信，选择稳定的 Wi-Fi 频段，避开信号拥堵的频段；使用信号质量监测工具，实时监测 Wi-Fi 信号强度和质量，当信号质量不佳时，自动切换到其他可用的 Wi-Fi 网络或提示用户调整设备位置；在应用中设置网络连接异常处理逻辑，当检测到网络连接中断时，自动尝试重新连接，并在连接成功后恢复数据传输。

4.2 权限管理问题

安卓应用在获取控制硬件所需权限时，可能会遇到权限申请被用户拒绝、系统权限限制等问题。例如，在申请访问串口权限时，如果用户拒绝授权，应用将无法与开发板进行通信，从而无法实现对 LED 灯泡的控制。

针对这一问题，首先在应用开发过程中，遵循安卓系统的权限申请规范，在应用启动时或首次需要相关权限时，向用户弹出清晰、明确的权限申请提示，说明申请该权限的目的和必要性，以提高用户授权的意愿。例如，在申请串口权限时，提示用户“为了实现通过手机控制 LED 灯泡，需要获取串口通信权限，请您授权”。如果用户拒绝授权，在应用中提供引导用户手动开启权限的功能，通过跳转到系统设置中的应用权限管理页面，让用户手动授予相应权限。同时，在应用的设置界面中，提供权限管理入口，方便用户随时查看和管理应用的权限。此外，对于一些特殊的系统级权限，如修改系统设置权限，可能需要通过特定的系统 API 或与设备厂商合作，以获取相应的权限支持。

4.3 硬件兼容性问题

不同型号的 LED 灯泡在电气特性、通信协议等方面可能存在差异，这可能导致开发板与部分 LED 灯泡不兼容，无法正常控制。例如，某些 LED 灯泡可能需要特定的控制信号电平或通信协议，而开发板默认的控制方式与之不匹配。

为解决硬件兼容性问题，在项目前期进行充分的市场调研，了解常见 LED 灯泡的型号和特性，选择兼容性较好的 LED 灯泡进行开发和测试。在硬件设计阶段，预留一定的硬件接口扩展能力，以便能够通过外接转换电路等方式，适配不同电气特性的 LED 灯泡。在软件层面，开发通用的驱动程序框架，通过配置文件或参数的方式，实现对不同通信协议和控制方式的 LED 灯泡的支持。当检测到新的 LED 灯泡接入时，应用能够自动识别其型号和特性，并加载相应的驱动配置，实现对其的控制。同时，建立硬件兼容性测试数据库，记录不同型号 LED 灯泡与开发板的兼容性情况，为后续的优化和改进提供参考。

五、预期成果

5.1 实现功能

- **LED 灯泡开关控制：**用户通过安卓应用界面上的开关按钮，能够轻松实现对 LED 灯泡的点亮和熄灭操作。当用户点击“打开”按钮时，应用向迅为开发板发送指令，开发板控制 GPIO 引脚输出高电平，LED 灯泡亮起；当用户点击“关闭”按钮时，应用发送相应指令，开发板控制 GPIO 引脚输出低电平，LED 灯泡熄灭。
- **状态反馈显示：**安卓应用能够实时获取 LED 灯泡的当前状态，并在应用界面上直观地展示给用户。通过与开发板的通信，应用接收开发板反馈的 LED 灯泡状态信息，若 LED 灯泡处于点亮状态，应用界面上的开关按钮显示为“关闭”状态，且按钮颜色或图标可进行相应变化，以提示用户灯泡已亮；若 LED 灯泡处于熄灭状态，按钮则显示为“打开”状态及对应的外观变化。
- **指令发送记录：**应用具备记录用户控制指令发送历史的功能，方便用户查看以往的操作记录。在应用内部，创建一个指令发送记录表，每次用户点击开关按钮发送控制指令时，将指令内容（打开或关闭）、发送时间等信息记录到表中。用户可在应用的特定界面中查看该记录列表，了解自己对 LED 灯泡的控制操作历史，便于追溯和管理。

5.2 性能指标

- **控制响应时间：**从用户在安卓应用上点击开关按钮，到 LED 灯泡实际状态发生改变的响应时间，控制在 500 毫秒以内。这一指标通过优化安卓应用与迅为开发板之间的通信协议、提高指令处理速度以及硬件电路的快速响应来实现。在软件层面，采用高效的数据传输算法和多线程处理机制，减少指令发送和接收的延迟；在硬件层面，选用性能优良的处理器和通信模块，确保数据能够快速传输和处理。
- **通信成功率：**安卓应用与迅为开发板之间的通信成功率达到 99% 以上。为实现这一目标，在通信过程中采用数据校验、重传机制以及信号增强等技术手段。例如，在数据发送端，对发送的数据添加校验码，接收端对接收到的数据进行校验，若校验失败则请求重发；同时，优化通信信号的强度和稳定性，减少信号干扰和丢失的情况，从而提高通信的可靠性。

- **系统稳定性：**在连续运行 24 小时的测试中，系统能够稳定工作，不出现异常崩溃、控制失效等问题。通过对系统全面的压力测试和稳定性测试，发现并修复潜在的软件漏洞和硬件兼容性问题。在软件方面，进行内存管理优化，避免内存泄漏和溢出等问题；在硬件方面，确保硬件设备的散热良好，避免因过热导致的工作异常，从而保障系统的长时间稳定运行。

六、进度安排

6.1 需求分析与方案设计阶段

- **第 1 周：**收集和分析项目需求，明确安卓应用控制 LED 灯泡点亮的功能要求、性能指标以及用户需求。研究相关技术资料，了解安卓应用开发、迅为开发板以及 LED 灯泡控制的技术原理和实现方法。
- **第 2 周：**根据需求分析结果，制定项目的总体方案，包括硬件设计方案和软件设计方案。确定硬件连接方式、开发板的选型以及安卓应用的架构和功能模块划分。撰写需求分析报告和系统设计文档，对项目的需求和设计进行详细的描述和说明，为后续的开发工作提供指导。

6.2 硬件搭建与软件开发阶段

- **第 3 周：**进行硬件搭建，按照硬件设计方案，将 LED 灯泡与迅为开发板进行连接，确保硬件连接正确、稳定。同时，搭建安卓开发环境，安装 Android Studio、JDK 等开发工具，配置开发环境变量，确保开发环境能够正常运行。
- **第 4 - 5 周：**进行开发板端驱动程序开发，根据开发板的硬件特性和 LED 灯泡的控制原理，开发相应的驱动程序，实现开发板对 LED 灯泡的硬件控制。在开发过程中，进行驱动程序的测试和调试，确保驱动程序能够正确地控制 LED 灯泡的亮灭。
- **第 6 - 7 周：**进行安卓应用开发，根据软件设计方案，实现安卓应用的用户界面和控制逻辑。创建用户界面，包括开关按钮等元素，实现用户与应用的交互功能。编写控制逻辑代码，实现通过应用向开发板发送控制指令的功能。在开发过程中，进行应用的功能测试和界面优化，确保应用的功能正常、界面友好。

6.3 测试与优化阶段

- **第 8 周：**对整个系统进行全面测试，包括功能测试、稳定性测试、兼容性测试等。功能测试主要检查安卓应用是否能够准确地控制 LED 灯泡的点亮和熄灭，以及应用的其他功能是否正常；稳定性测试主要测试系统在长时间运行过程中的稳定性，

检查是否会出现异常崩溃等问题；兼容性测试主要测试系统在不同的安卓设备上的兼容性，确保系统能够在各种设备上正常运行。

- **第 9 周：**根据测试结果，对系统进行优化和改进。针对测试中发现的问题，如控制响应时间过长、通信不稳定等，进行分析和调试，采取相应的优化措施，如优化代码、调整硬件参数等，提高系统的性能和稳定性。对系统的功能和界面进行进一步的优化，提升用户体验。

6.4 项目验收阶段

- **第 10 周：**进行项目验收，提交项目的相关文档，包括需求分析报告、系统设计文档、测试报告、用户手册等，展示项目的成果和实现的功能。根据项目的目标和验收标准，对项目进行评估和验收，检查项目是否达到预期的目标和要求。如果项目通过验收，则项目结束；如果项目未通过验收，则根据验收意见进行整改和完善，直到项目通过验收为止。

七、参考文献

- [1] 明日科技. Android 从入门到精通 [M]. 清华大学出版社, 2020.
- [2] 李宁. 疯狂 Android 讲义 [M]. 电子工业出版社, 2021.
- [3] 迅为电子. iTOP-XX 开发板用户手册 [Z]. 迅为电子有限公司, 2022.
- [4] 韦东山. 嵌入式 Linux 应用开发完全手册 [M]. 人民邮电出版社, 2019.
- [5] 孙鑫. Java 核心技术 [M]. 电子工业出版社, 2020.